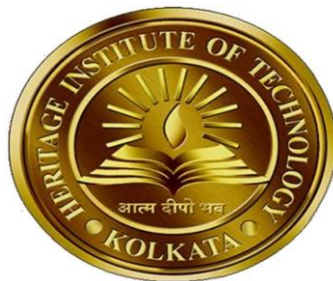


SafeSign: An End-to-End Handwritten Signature Verification System

Project Report



Submitted by:

Sachin Kumar, Roll no: 2251074, Autonomy Roll no: 1262201117

Sumanta Rakshit, Roll no: 2251079, Autonomy Roll no: 12622001168

Sayandeep Biswas, Roll no: 2251061, Autonomy Roll no: 12622001135

Aaditya Kumar Chaudhury, Roll no: 2251078, Autonomy Roll no: 12622001001

Under the supervision of:

Prof. Anurina Tarafdar

Department of Computer Science and Engineering

In partial fulfilment for the award of the degree

Of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

HERITAGE INSTITUTE OF TECHNOLOGY,

KOLKATA, 2025

**HERITAGE INSTITUTE OF TECHNOLOGY,
KOLKATA**

**MAULANA ABUL AZAD UNIVERSITY OF
TECHNOLOGY**



BONAFIDE CERTIFICATE

Certified that this project report “SAFESIGN: END-TO-END HANDWRITTEN SIGNATURE VERIFICATION SYSTEM” is the bonafide work of “Mr. Sachin Kumar, Mr. Sumanta Rakshit, Mr. Aaditya Kumar Chaudhury, Mr. Sayandeep Biswas” who carried out the project under my supervision.

Prof.(Dr.) Subhashis Majumder
Head of the Department
Computer Science and Engineering

Prof. Anurina Tarafdar
Assistant Professor
Computer Science and Engineering

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of the people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all effort with success.

We would take this opportunity to thank our Prof. (Dr.) BASAB CHAUDHURI, for allowing us to form a group of four people and for providing us with all necessary facilities to make our project work and worth.

We will be thankful to our head of the department Prof. Dr. Subhashis Majumder, for constantly supporting us and giving us valuable insights. His word of encouragement motivated us to achieve our goals and impetus to excel.

We are grateful to our project guide Prof Anurina Tarafdar for the guidance, inspiration and constructive suggestion that helped us in the partial fulfillment for the award of the degree.

Sachin Kumar

Sumanta Rakshit

Aaditya Kumar Chaudhury

Sayandeep Biswas

TABLE OF CONTENTS

<u>Abstract</u>	5
1. <u>Introduction</u>	6
2. <u>Literature Survey</u>	8
3. <u>Objective</u>	14
4. <u>Technologies and Algorithms Used</u>	15
5. <u>Input and Output of the Project</u>	22
6. <u>Approach and Methodology</u>	26
6.1 <u>Phase 1: Exploratory Data Analysis (EDA)</u>	
6.2 <u>Phase 2: Data Preprocessing Pipeline</u>	
6.3 <u>Phase 3: Siamese Network Implementation</u>	
6.4 <u>Phase 4: Verification and Similarity Logic</u>	
7. <u>Results</u>	33
8. <u>Conclusion</u>	37
9. <u>Limitation of the project and Future Scope</u>	38
10. <u>References</u>	40

Abstract of our Project:

The shortcomings of the current handwritten signature verification methods are the driving force behind this effort. Traditional verification systems often rely on manual inspection or simple pattern-matching techniques. While these approaches offer basic validation, they fail to capture the deeper structural variations present in genuine signatures and the subtle inconsistencies found in skilled forgeries.

Our objective is to enhance the signature verification process by leveraging deep metric learning through a **Siamese Neural Network**. We aim to address the following drawbacks of existing systems:

Lack of Robust Forgery Detection:

Conventional models struggle to differentiate between genuine signatures and skilled forged signatures, especially when intra-class variations are high.

High Dependence on Manual Verification:

Many verification workflows still require human experts, making the process slow, inconsistent, and prone to subjective errors.

Poor Generalization Across Datasets:

Signature styles vary significantly across individuals and regions. Existing systems often fail to adapt well when trained on limited or single-source datasets.

Our project aims to bridge the gap between accuracy and real-world applicability in signature verification systems. By incorporating a **Siamese Neural Network** architecture trained on combined datasets (**GPDS150 and CEDAR**) and supported with an optimized preprocessing pipeline, we aspire to build a system that can reliably classify signatures as genuine or forged. Furthermore, our ultimate goal is to integrate this intelligent verification model into a web-based platform, enabling secure, fast, and automated validation of handwritten signatures for real-world applications such as banking and legal documentation.

1. Introduction

A signature verification system is a specialized software solution or algorithm designed to determine the authenticity of handwritten signatures. The primary objective of such systems is to assist organizations in validating user identity, ensuring document security, and preventing fraud. These systems are widely used in financial institutions, legal services, and administrative processes where signature-based authorization plays a critical role. By analyzing signature characteristics such as stroke shape, pressure patterns, and writing behavior, these systems improve accuracy and reliability in verification, thereby enhancing overall security.

What is a Handwritten Signature Verification System?

A handwritten signature verification system is a biometric authentication method used to compare a user's *query signature* with a *reference signature* to determine whether both belong to the same individual. Unlike traditional rule-based methods, modern verification systems use machine learning and deep learning techniques to understand the unique features of a person's signature, including slant, curvature, speed, and pen dynamics. This approach surpasses conventional manual checking or simple pattern-matching systems, offering a more accurate, consistent, and automated verification process.

Key Components of a Signature Verification System

1. Signature Acquisition:

The process begins by collecting both genuine and forged signatures from multiple users. These samples serve as input for training and validating the verification model.

2. Preprocessing:

Before analysis, signatures undergo several preprocessing steps such as grayscale conversion, binarization, noise removal, and aspect-ratio-preserving resizing. These steps ensure uniformity and help the model focus on essential stroke patterns.

3. Feature Extraction:

Feature extraction involves identifying key aspects of the signature, such as contour shape, edges, stroke direction, and pixel intensity distribution. Deep learning networks automatically learn these features from training data.

4. Machine Learning / Deep Learning Model:

Advanced models such as Siamese Neural Networks are used to compare two signature images. These models generate high-dimensional feature embeddings and compute similarity scores to detect whether a signature is genuine or forged.

5. Pair Generation:

The system creates *positive pairs* (genuine–genuine) and *negative pairs* (genuine–forged) to train the model on how signatures of the same person differ from those of forgeries.

6. Similarity Measurement:

Using distance metrics such as Euclidean distance, the system determines how closely the query signature matches the reference signature. A threshold value is used for final classification.

7. Real-Time Verification:

Once deployed, the system can analyze a user's signature in real time and instantly provide results, making it suitable for banking transactions, document validation, and digital workflows.

8. Integration with Web Applications:

Signature verification systems can be integrated into online platforms, enabling secure authentication processes through web interfaces. This allows users to upload signatures and receive verification results instantly.

What is a Siamese Neural Network?

A Siamese Neural Network is a type of deep learning architecture designed to learn similarity between two inputs. Instead of classifying signatures into predefined categories, the network determines how similar two signature embeddings are. This makes SNNs ideal for signature verification, as they can handle natural variations in a person's handwriting while effectively detecting forged samples.

By combining deep learning with robust preprocessing and intelligent pair generation, our system aims to provide a secure, fast, and highly accurate handwritten signature verification solution. The ultimate goal is to integrate this verification capability into a web-based application, offering a seamless, automated, and user-friendly authentication experience.

2. Literature Survey

Over the past decade, offline handwritten signature verification has transitioned from handcrafted feature engineering to sophisticated deep-learning approaches. This section surveys key research contributions, model architectures, strengths, and limitations across the field. The aim is to identify persistent challenges and position the *SafeSign* project within the broader research landscape.

2.2 Foundational Deep-Learning Approaches

2.2.1 SigNet (2017) - Siamese CNN for Writer-Independent Verification

- **Contributions:** Introduces deep metric learning with contrastive loss, replacing handcrafted features.
 - **Strengths:**
 - High performance on CEDAR, GPDS-300, GPDS-Synthetic, BHSig260
 - Strong cross-script generalization
 - **Limitations:**
 - Requires large-scale training data
 - Does not account for noise, scanning variability, or deployment constraints
-

2.3 Representation Learning and Generative Models

2.3.1 Feature-Disentangling Variational Autoencoder (2024)

- **Contributions:** Learns latent representations without skilled forgeries, using disentangling loss + VAE.
- **Strengths:**

- Strong EER on MCYT-75 and GPDS-Synthetic
 - Latent features effectively classified by SVM
 - **Limitations:**
 - Limited dataset diversity
 - Uncertain robustness to skilled forgeries
 - Real-world applicability remains unclear
-

2.4 Adaptive and Stream-Based Verification Models

2.4.1 Stream-Based Signature Verification (2024)

- **Contributions:** Treats signature verification as a data-stream task with incremental model updates.
 - **Strengths:**
 - EER improvement from ~15% to ~7.7%
 - Outperforms SVM baselines
 - **Limitations:**
 - Computational overhead
 - Vulnerability to concept drift
 - Sensitivity to noise and evolving writing styles
-

2.5 Deep CNN Feature Learning Approaches

2.5.1 Hafemann et al. - Deep CNN with Multi-Task Learning

- **Contributions:** Writer-independent deep features; forgery-detection multi-task objective.
- **Strengths:**
 - Achieves 1.72% EER on GPDS-160
 - Strong generalization to CEDAR and MCYT

- **Limitations:**
 - Requires skilled forgeries during training
 - Uses user-specific thresholds
-

2.6 Insights from Online Signature Verification

2.6.1 TS-GATR (Graph Attention Transformer, 2025)

- **Contributions:** Models temporal + spatial handwriting dynamics using graph attention.
 - **Strengths:**
 - Captures rhythm, stroke patterns → helpful for skilled forgery detection
 - **Limitations:**
 - Computationally expensive
 - Reduced performance for finger-based signatures
 - **Relevance:** Demonstrates advantages of temporal data not available in offline systems.
-

2.7 Verification in Real-World Noisy Document Settings

2.7.1 SignForensics Framework

- **Contributions:** Designed for “in-the-wild” signatures extracted from noisy documents.
- **Pipeline:** YOLOv10 (detection) → CycleGAN (artifact removal) → SigNet + CapsNet (verification)
- **Strengths:**
 - Exceptional performance (100% on CEDAR; >94% on Bengali/Hindi datasets)
 - Highly robust preprocessing

- **Limitations:**
 - High dependence on preprocessing modules
 - Increased system complexity
-

2.8 Cross-Dataset Generalization

2.8.1 Signature Forgery Detection (2025)

- **Contributions:** ResNet-34 Siamese network with shell-based preprocessing for domain robustness.
 - **Strengths:**
 - Improved cross-dataset stability
 - **Limitations:**
 - Still exhibits domain shift issues
 - Reduced peak accuracy due to heavier normalization
 - **Key Insight:** Cross-dataset variance remains a major open challenge.
-

2.9 Lightweight Verification Models

2.9.1 SqueezeNet-Based Signature Verification (2025)

- **Contributions:** Demonstrates use of compact CNNs for efficient verification.
 - **Strengths:**
 - 98–99% accuracy on CEDAR
 - Extremely low parameter count → ideal for mobile deployment
 - **Limitations:**
 - Struggles with skilled forgeries
 - Limited writer-independent experiments
-

2.10 Hybrid CNN - Transformer Architectures

2.10.1 Hybrid Vision Transformer (2024)

- **Contributions:** Combines CNN-based local features with global transformer attention.
 - **Strengths:**
 - Excellent performance on CEDAR and BHSig
 - **Limitations:**
 - Poor performance on UTSig-Persian (~74%)
 - High computational cost
 - Unknown effectiveness on skilled forgeries
 - **Insight:** Script variation significantly influences accuracy.
-

2.11 Classical Machine-Learning and Transfer-Learning Approaches

2.11.1 Ensemble SVM System (2023)

- **Contributions:** Uses handcrafted features (edge histograms, Radon transforms, fractal dimensions).
- **Strengths:**
 - ~97% accuracy on CEDAR and GPDS-Synthetic
 - Lightweight and computationally efficient
- **Limitations:**
 - Poor scalability
 - Underperforms on large or diverse datasets

2.11.2 MobileNetV2 + Feature Selection (2024)

- **Strengths:**
 - High accuracy (~97%)
- **Limitations:**

- Tested without forged samples → lacks real-world validation
- Limited generalizability

2.12 Common Research Gaps

Across the literature, several persistent gaps emerge:

- Limited cross-dataset generalization
 - Heavy reliance on clean, well-preprocessed inputs
 - Insufficient evaluation on skilled forgeries
 - Weak writer-independent performance
 - Lack of lightweight, real-time deployable systems
 - Script and cultural variability affecting accuracy
 - Overly complex pipelines (e.g., reliance on YOLO + GANs)
-

2.13 Positioning the SafeSign Project

The *SafeSign* project aims to systematically address these gaps by developing a:

- Dataset-agnostic Siamese Network
- Unified, reliable preprocessing pipeline
- Training paradigm combining multiple datasets
- Real-time deployable web-based verification system
- Model robust to domain shift, noise, and skilled forgeries

3. Objectives

The objective of this project is to develop an intelligent handwritten signature verification system capable of accurately identifying whether a given signature is genuine or forged. To fulfill this overall objective, we consider five sub-problems which are as follows: –

1. **Signature Preprocessing & Standardization:**

We will design a robust preprocessing pipeline that converts raw signature images into clean and standardized inputs. This includes grayscale conversion, binarization, background removal, inversion, and aspect-ratio–preserving resizing to ensure uniform representation of all signatures across datasets.

2. **Feature Extraction Using Siamese Neural Network:**

A Siamese Neural Network architecture will be used to learn high-dimensional feature embeddings from pairs of signature images. The twin network structure enables the model to extract discriminative features that distinguish genuine signatures from forged ones, even when writing variations exist.

3. **Similarity Measurement & Classification:**

We will compute similarity between signature pairs using Euclidean distance. The model will classify signatures as *genuine* or *forged* based on whether the distance falls below or above a predefined threshold. This metric-learning approach ensures accurate verification across a diverse population of users.

4. **Dataset Integration & Model Training:**

We will train the system using a combination of benchmark offline signature datasets such as GPDS150 and CEDAR to increase diversity and robustness. Positive (genuine–genuine) and negative (genuine–forged) pairs will be generated to effectively train the Siamese network using supervised contrastive learning.

5. **Real-Time Verification & Web Deployment:**

Our system will be integrated into a web-based interface that allows users to upload both reference and query signatures. The model will perform real-time verification and display results instantly. This dynamic and interactive deployment ensures practical usability in real-world scenarios such as banking and document authentication.

4. Technologies and Algorithms Used

This section outlines the core technological stack, theoretical frameworks, and specific algorithmic choices employed in the development of the "SafeSign" End-to-End Handwritten Signature Verification System. The system leverages state-of-the-art Deep Learning architectures, specifically Siamese Neural Networks, alongside traditional Computer Vision algorithms for preprocessing.

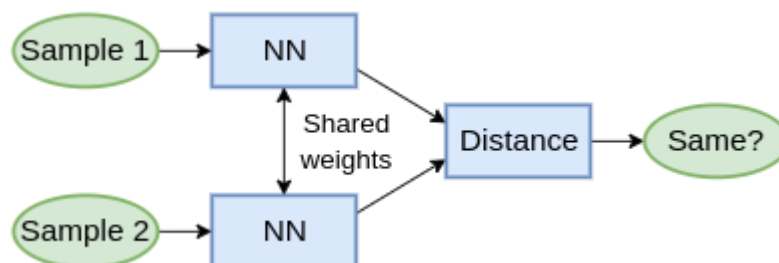
4.1. Deep Learning Framework: Siamese Neural Networks

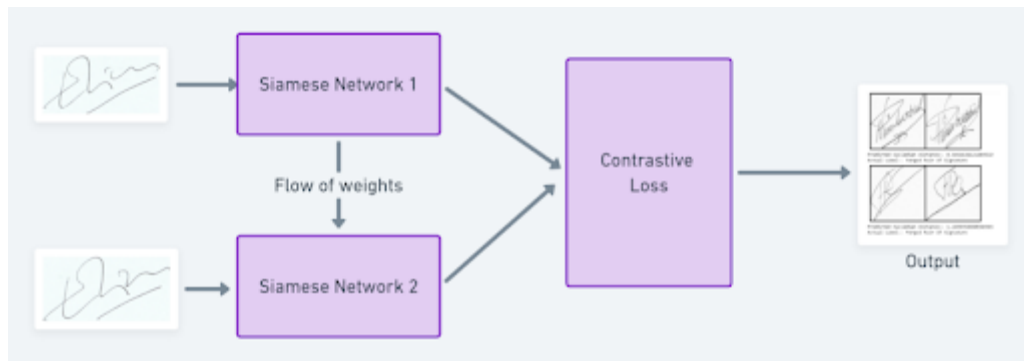
The core engine of the SafeSign system is a **Siamese Neural Network (SNN)**. Unlike standard classification models that categorize an image into a specific class (e.g., "Cat" or "Dog"), the SNN is designed for **One-Shot Learning**. It learns to calculate a high-dimensional feature vector (embedding) for two different input images. Siamese network networks are used to find the similarity of the inputs by comparing their **feature vectors**.

4.1.1. Architecture Overview

The architecture consists of two identical subnetworks (twin networks) that share the exact same weights and parameters.

- **Input:** The network accepts a pair of signature images (X1, X2).
- **Feature Extraction:** Both images are passed through the twin networks simultaneously to generate two feature embeddings E1 and E2.
- **Weight Sharing:** $W_{net1} = W_{net2}$. This guarantees that two extremely similar images will map to very similar feature vectors in the latent space because they are processed by the same function.

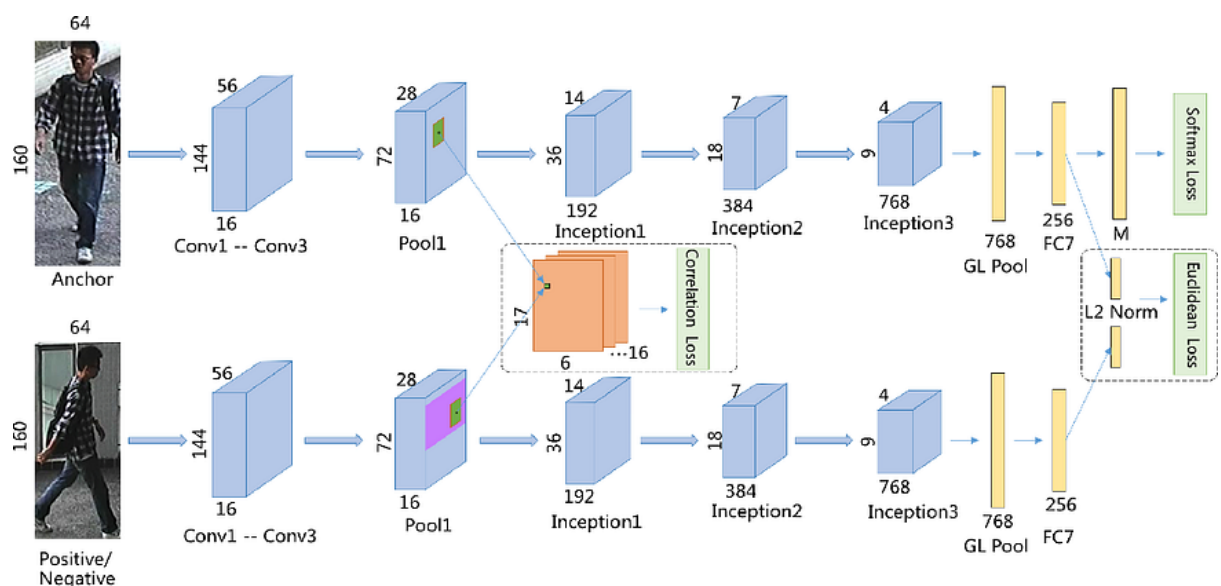




4.1.2 step-by-step approach of how siamese network architecture works:

We have two images and we want to compare and see if they are similar or dissimilar pairs

- The first subnetwork takes an image (A) as input and passes through convolutional layers and fully connected layers, we get a vector representation of the image
- Again pass the second image(B) through a network that is exactly the same with the same weights and parameters.
- Now we have two encodings $E(A)$ and $E(B)$ from the respective images, we can compare these two to know how similar the two images are. If the images are similar then the encodings will also be quite similar.
- We will measure the distance between these two vectors and if the distance between these is small then the vectors are similar or of the same classes and if the distance between is larger then the vectors are different from one another, based on the score.

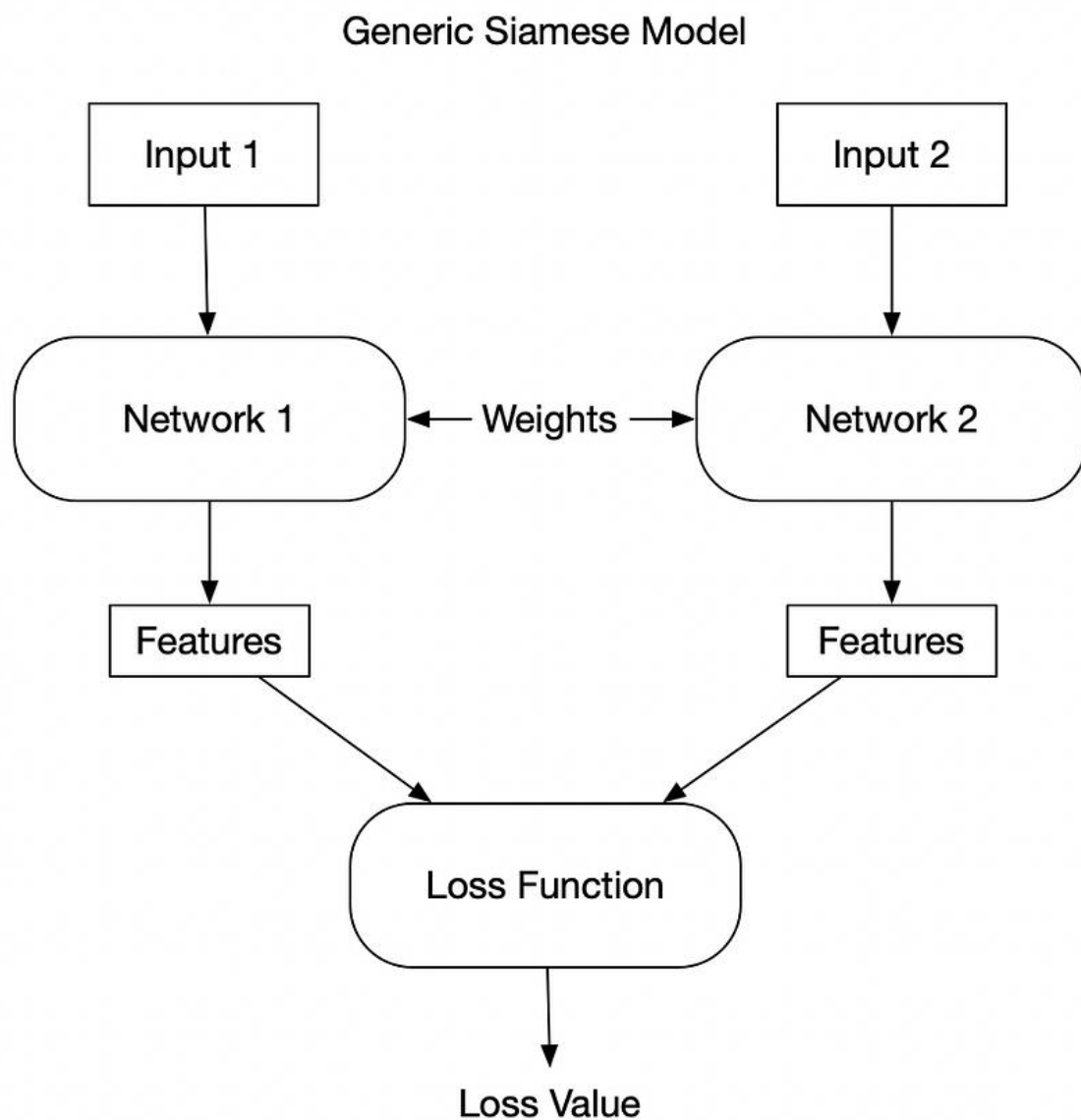


4.1.3 Loss function in siamese networks

Let us go through two main loss functions of the siamese network which are contrastive loss and Triplet loss.

- **Contrastive loss function**

Siamese network is not to perform classification on input images, but to differentiate between input images. So, classification loss functions like cross-entropy loss, would not be the best fit.



Instead, this siamese network architecture is better suited to use a **contrastive function**.

This function just evaluates how well the siamese network is able to distinguish between the given image pairs.

The contrastive loss function formulae are as follows:

$$(1 - Y)\frac{1}{2}(D_W)^2 + (Y)\frac{1}{2}\{max(0, m - D_W)\}^2$$

where **D_W** is defined as the Euclidean distance between the outputs of the sister networks.

Mathematically the formulae of euclidean distance are:

$$\sqrt{\{G_W(X_1) - G_W(X_2)\}^2}$$

Y is either 1 or 0. If the first image and second image are from the same class, then the value of Y is 0, otherwise, Y is 1

max() is a function denoting the higher value between 0 and m-D_W.

m is a margin value that is greater than 0. Having a margin indicates that dissimilar pairs that are beyond this margin will not contribute to the loss.

- **Triplet loss function**

Triplet loss will allow our model to map two similar images close and far from dissimilar sample image pairs.

This approach is done by using triplet constituting:

1. Anchor Image — This is a sample image.

2. Positive Image — This is just another variation of the anchor image.

This helps the siamese network model learn the similarities between the two images.

3. Negative Image — This is a different image from the above two similar image pairs.

This helps our model learn dissimilarities with anchor images.

Now, to increase the distance between similar and dissimilar pairs output vector, and to map similar images close to one another, there is a term known as margin. Margin increases the separation between our similar and dissimilar vector, and also eliminate the output of any trivial solution.

This similarity or dissimilarity is measured by the distance between two vectors using **L2 distance and cosine distance**.

Here is our loss function, where a is the anchor image, p is the positive image and n is the negative images:

$$\text{loss}(a, p, n) = \max(0, d(a, p) - d(a, n) + \text{margin})$$

4.1.3. Similarity Metric: Euclidean Distance

To determine the authenticity of a signature, the system quantifies the similarity between the two embeddings (E_1 , E_2). SafeSign employs the **Euclidean Distance** metric.

The distance d between two embedding vectors is calculated as:

$$d(E_1, E_2) = \|E_1 - E_2\|_2 = \sqrt{\sum_{i=1}^n (E_{1,i} - E_{2,i})^2}$$

- If $d(E_1, E_2) < \text{threshold}$, the signatures are classified as a **Positive Pair (Genuine)**.
- If $d(E_1, E_2) > \text{threshold}$, the signatures are classified as a **Negative Pair (Forged)**.

4.1.4. Pros and Cons of Siamese Networks

The main advantages or pros of siamese networks are,

1. **Robustness to class Imbalance:** Due to one-shot learning, a few images (very little data for training data) for the same class is sufficient for siamese networks to classify those images in the future
2. **Ensemble with one of the classifier algorithms:** As its learning mechanism is different from Classification algorithms, ensembling the siamese networks with a classifier can do much better than average two correlated Supervised models (e.g. GBM & RF classifier algorithms)
3. **Semantic Similarity:** trained siamese network focuses on learning embeddings (in the deep neural networks) that place the same classes close together. Hence, can learn **semantic similarity**.

The downsides or cons of the siamese networks can be,

1. **Requires more training time than traditional neural network architectures and machine learning algorithms:** Siamese Networks involve quadratic pairs to learn

from which is slower than the normal traditional type of machine learning, the neural network learns fast than the siamese network.

2. **Doesn't output probabilities:** Since training of siamese networks involves pairwise learning, it does not output the probabilities of the prediction, but the distance (using a distance formula like euclidean distance) from each class which is between 0 to 1.

4.2. Backbone Convolutional Neural Networks (CNNs)

To extract meaningful features from the raw signature images, we employed three distinct base CNN architectures. This comparative approach allowed us to balance speed, depth, and accuracy.

4.2.1. Custom CNN

A lightweight, custom-designed Convolutional Neural Network was implemented to serve as a baseline.

- **Purpose:** Optimized specifically for faster inference times, making it suitable for environments with limited computational resources.
- **Structure:** It typically consists of sequential convolutional layers followed by Max Pooling and ReLU activation functions, ending in a fully connected layer that outputs the feature embedding.

4.2.2. ResNet-18 (Residual Networks)

We incorporated ResNet-18 to address the "vanishing gradient" problem often faced in deeper networks.

- **Mechanism:** ResNet introduces "skip connections" (or identity shortcuts) that allow the gradient to flow through the network more easily during backpropagation.
- **Mathematical Representation:**

$$y = \mathcal{F}(x, \{W_i\}) + x$$

Where $\mathcal{F}$ is the residual mapping to be learned.

- **Benefit:** This architecture is highly suitable for deep feature extraction, capturing subtle stroke dynamics in signatures without performance degradation.

4.2.3. EfficientNet-B0

The third architecture utilized is EfficientNet-B0, which focuses on efficiency by uniformly scaling network width, depth, and resolution.

- **Compound Scaling:** Unlike conventional scaling (which only increases one dimension), EfficientNet uses a compound coefficient ϕ to scale all dimensions.
- **Advantage:** It provides a superior balance between model size and verification accuracy.

4.3. Image Processing Algorithms

Before feature extraction, raw images undergo rigorous preprocessing using traditional computer vision algorithms to ensure data consistency.

4.3.1. Otsu's Thresholding (Binarization)

To isolate the signature from the paper background, we utilize Otsu's Binarization method.

- **Algorithm:** This algorithm assumes the image contains two classes of pixels (foreground ink and background paper) and calculates the optimum threshold t^* that minimizes the weighted within-class variance (σ_w^2):

$$\sigma_w^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t)$$

- **Output:** A binary image where the signature is clearly separated from the background.

4.3.2. Sparse Processing (Inversion)

Following binarization, the image is inverted.

- **Operation:**

$$Pixel_{new} = 255 - Pixel_{old}.$$

- **Result:** Ink is mapped to white (255) and the background to black (0). This is critical for computational efficiency, as the network can focus on non-zero values (the sparse strokes) rather than the vast background.

4.3.3. Aspect-Ratio Preserved Resizing

Standard resizing distorts signature geometry (e.g., compressing a long signature into a square). To prevent this, we use a padding strategy.

- **Method:** The image is scaled to fit within a 224 X 224. box while maintaining its original aspect ratio. The remaining areas are padded with the background color (black).
- **Significance:** This preserves the unique slant and width of the signature, which are critical features for distinguishing forgeries.

4.4. Software and Implementation Stack

The following table summarizes the technological environment used for development and deployment.

Component	Technology / Library	Description
Language	Python	Primary programming language for modeling and backend logic.
ML Framework	PyTorch	Used for building and training the Siamese Networks.
Image Processing	OpenCV / PIL	Used for Grayscale conversion, Otsu's thresholding and resizing.
Web Integration	Web Framework (e.g., FastAPI, Django)	Used to integrate the model into a web application for real-world deployment ¹⁶ .

5. Input and Output of the Project

This section details the data flow within the SafeSign system, defining the specific nature of the inputs required to initiate the verification process and the granular outputs generated by the model. The system operates on a "One-Shot Learning" paradigm, requiring pairs of inputs rather than single samples for classification.

5.1. System Input

The input to the SafeSign system is multifaceted, consisting of raw image data, structured dataset pairs for training, and pre-processed tensors for the neural network.

5.1.1. User Input

In a real-world deployment scenario (integrated with a web application), the user provides two distinct images for comparison:

1. **Reference Signature:** A verified, pre-existing signature of the known user (the ground truth).
2. **Query Signature:** The questionable signature that needs verification (e.g. from a bank cheque or document).

5.1.2. Training Data Input (Development Phase)

During the training phase, the model does not ingest single images in isolation. Instead, it requires **Siamese Pairs** to learn the concept of similarity. We constructed these pairs from a combined dataset of GPDS150 and CEDAR.

The inputs are categorized into two specific label types:

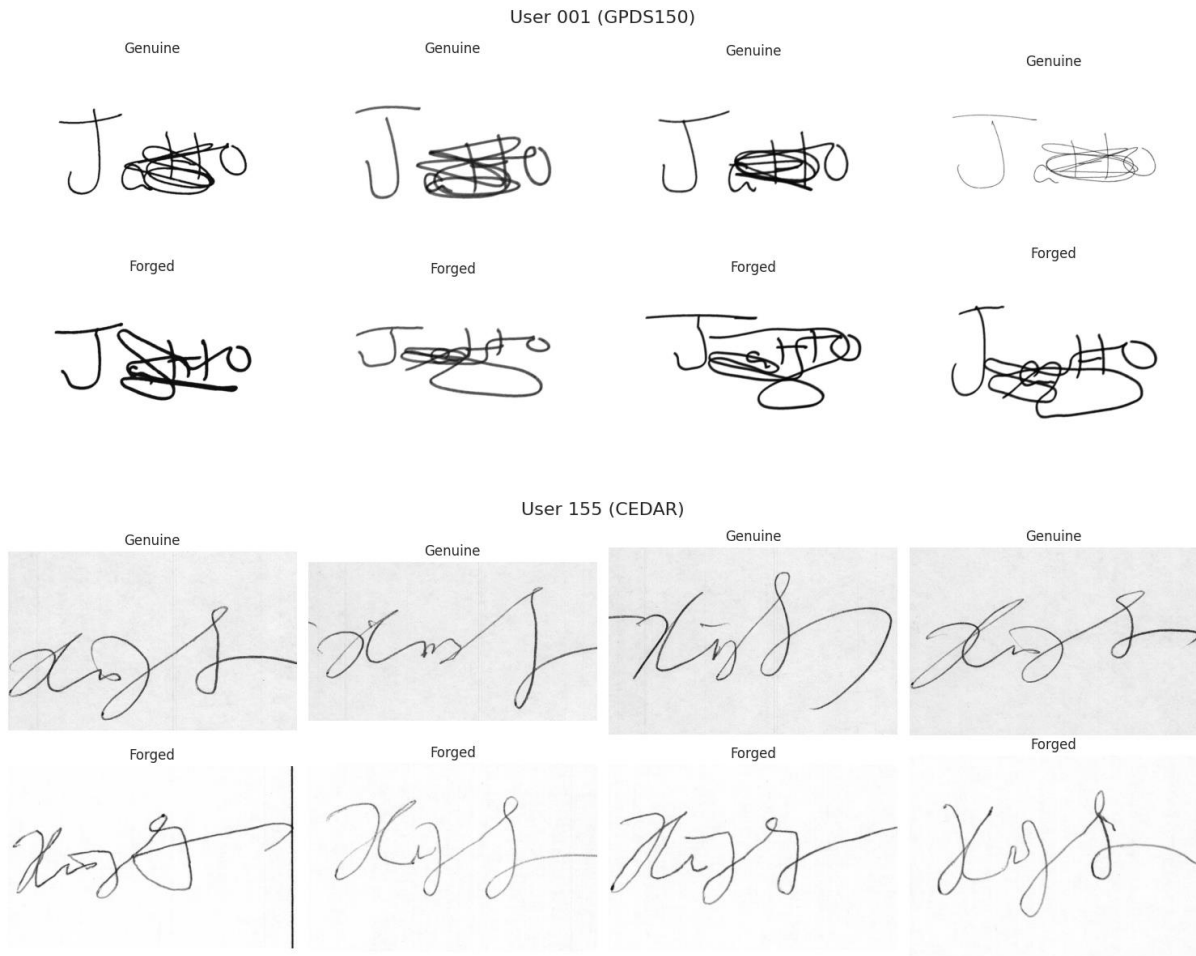
- **Positive Pairs (Label 1):** Two distinct signatures belonging to the same user (Genuine - Genuine).
- **Negative Pairs (Label 0):** A genuine signature paired with a forged signature of the same user (Genuine - Forged).

Dataset Specifications:

The following table summarizes the raw data input statistics utilized to train the model:

Dataset	No. of Users	Samples per User	Description
GPDS150	150	16 Genuine, 16 Forged	Large-scale dataset providing high variance in signature styles.
CEDAR	55	24 Genuine, 24 Forged	Standard benchmark dataset for signature verification.
Total	205 Users	7440 Images	Combined total images processed.

While datasets like BHSig250 (Bengali and Hindi) are available, they are reserved for future training iterations.



5.1.3. Pre-processed Input Dimensions

Before entering the Siamese Network, the raw input images are transformed into standardized tensors. The network cannot accept arbitrary image sizes; therefore, all inputs are converted to:

- **Dimensions:** 224 x 224 pixels.
- **Channels:** 1 (Grayscale/Binary).
- **Value Range:** Normalized pixel intensity (0 to 255) where ink is inverted to 255 (White) and background to 0 (Black).

5.2. System Output

The output of the SafeSign system occurs in three stages: the latent feature extraction, the quantitative distance calculation, and the final qualitative classification.

5.2.1. Feature Embeddings

The immediate output from the twin CNNs is not a class label, but a **high-dimensional feature vector** (embedding).

- For Input A (Reference), the network outputs Embedding EA.
- For Input B (Query), the network outputs Embedding EB.

These vectors represent the abstract features of the signature (stroke curvature, pressure points, geometric topology) in a multi-dimensional space.

5.2.2. Euclidean Distance

The system calculates the dissimilarity between the two embeddings using Euclidean Distance. This results in a scalar value representing the "distance" between the signatures.

- **Low Distance:** Indicates high similarity (signatures are likely from the same person).
- **High Distance:** Indicates low similarity (signatures are likely different or forged).

5.2.3. Final Classification

The final binary output is determined by comparing the calculated distance against a pre-defined **threshold**.

- **Threshold Value:** Currently set to **0.5**.

The decision logic is formalized as follows:

$$\text{Output Class} = \begin{cases} \text{Genuine (Authentic)} & \text{if } ||E_A - E_B||_2 < 0.5 \\ \text{Forged (Fake)} & \text{if } ||E_A - E_B||_2 \geq 0.5 \end{cases}$$

This binary classification is the final result presented to the user via the web application interface.

6. Approach and Methodology

This section talks about the systematic approach adopted for the development of SafeSign. The methodology is divided into four distinct phases: Exploratory Data Analysis (EDA), Data Preprocessing, Model Architecture Implementation, Verification Logic. Each phase was critically designed to address the challenges of signature verification, particularly the high variance in handwriting styles and the subtlety of skilled forgeries.

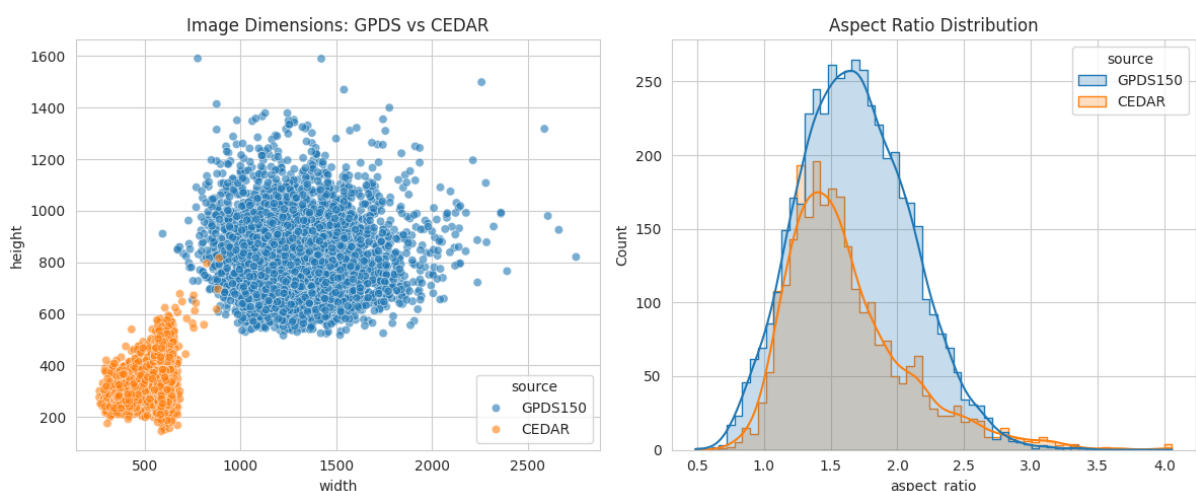
6.1. Phase 1: Exploratory Data Analysis (EDA)

Before commencing model training, we conducted a rigorous statistical analysis of the dataset characteristics to understand the variance in signature styles and image quality across the GPDS150 and CEDAR datasets. This analysis was crucial for establishing baseline performance metrics and informing our preprocessing strategy.

6.1.1. Resolution and Aspect Ratio Analysis

Handwritten signatures vary significantly in shape; some are tall and narrow, while others are wide and short.

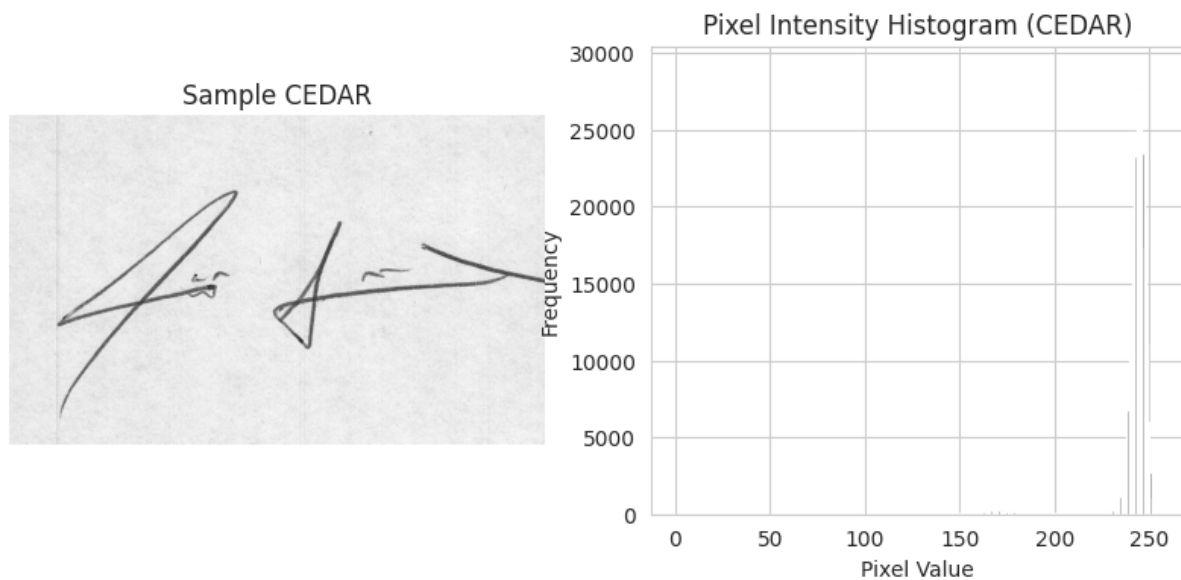
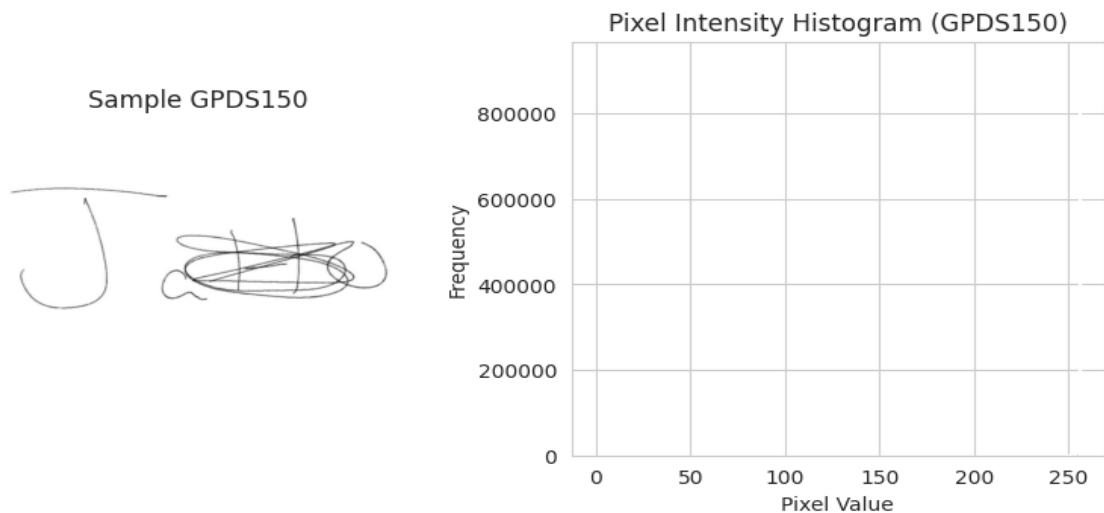
- **Analysis:** We calculated the height/width distributions of all samples in the dataset.
- **Insight:** Direct resizing of images to a square format (e.g. 224x224) without preserving aspect ratio would distort the signature's geometry (slant and stroke width), which are critical features for verification.
- **Decision:** This analysis directly informed our decision to use an "Aspect-Ratio Preserved Resizing" strategy with padding, rather than simple stretching.

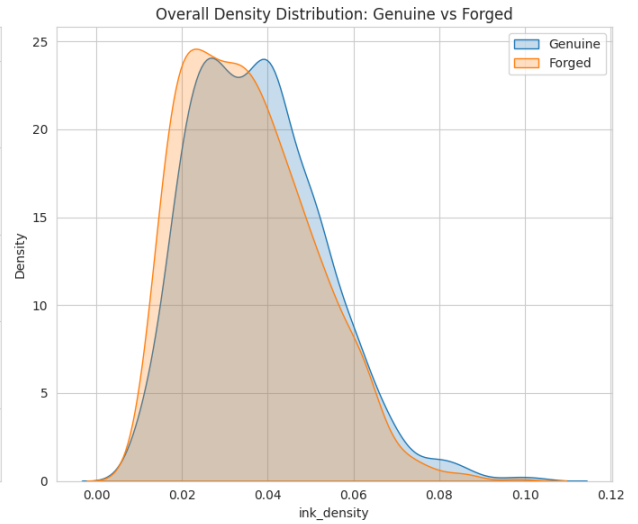
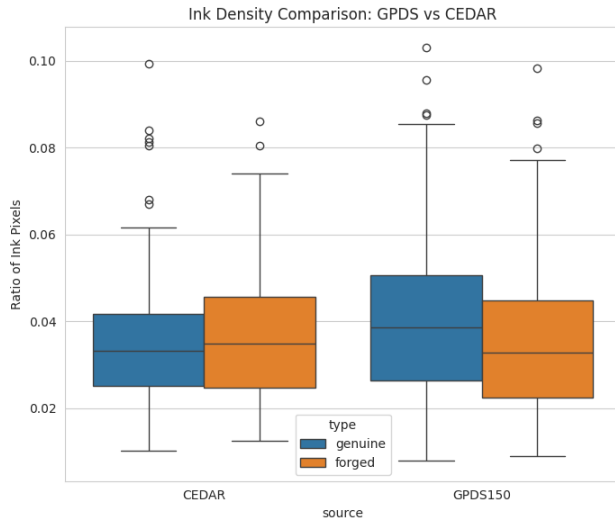


6.1.2. Pixel Intensity and Ink Density

We compared the pixel histograms of genuine signatures against forged samples to identify distinguishing features at the raw pixel level.

- **Observation:** Genuine signatures often exhibit more fluid ink density, representing natural hand movement.
- **Contrast:** Skilled forgeries often display "hesitations" or irregular ink deposits caused by the forger drawing slowly or lifting the pen, which results in different pixel intensity distributions.

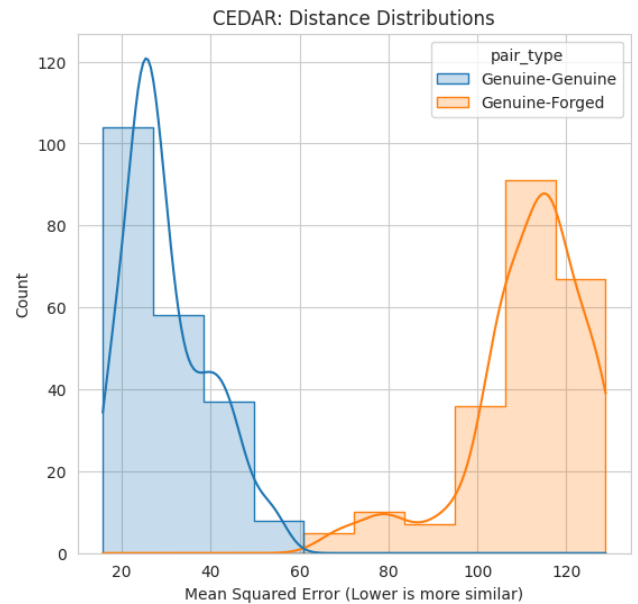
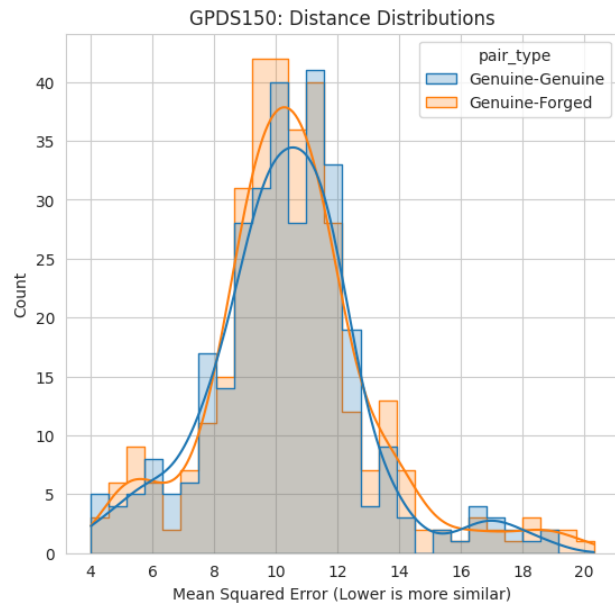




6.1.3. Baseline Distance Distribution

To establish a performance baseline, we calculated the initial Euclidean distances between pairs using raw pixel values (before any deep learning feature extraction).

- **Genuine-Genuine Pairs:** Showed lower distance on average but with high variance due to alignment issues.
- **Genuine-Forged Pairs:** Showed significant overlap with genuine pairs in raw pixel space, confirming that simple pixel comparison is insufficient and Deep Learning embeddings are necessary.



6.2. Phase 2: Data Preprocessing Pipeline

The raw images undergo a four-stage preprocessing pipeline designed to standardize inputs and isolate the "signal" (ink) from the "noise" (paper texture/background).

Step 1: Grayscale Conversion

The first step involves removing color channels (RGB) from the input images.

- **Rationale:** The color of the ink (blue vs. black) is irrelevant for verification; only the stroke pattern and topology matter. Reducing the image to a single channel reduces computational load.

Step 2: Binarization (Otsu's Thresholding)

We applied Otsu's Thresholding to automatically isolate the signature from the paper background.

- **Mechanism:** Otsu's method calculates a global threshold that minimizes intra-class variance, effectively separating the image into foreground (ink) and background.

Step 3: Inversion (Sparse Processing)

Following binarization, the pixel values are inverted.

- Transformation:

$$P_{new} = |255 - P_{old}|$$

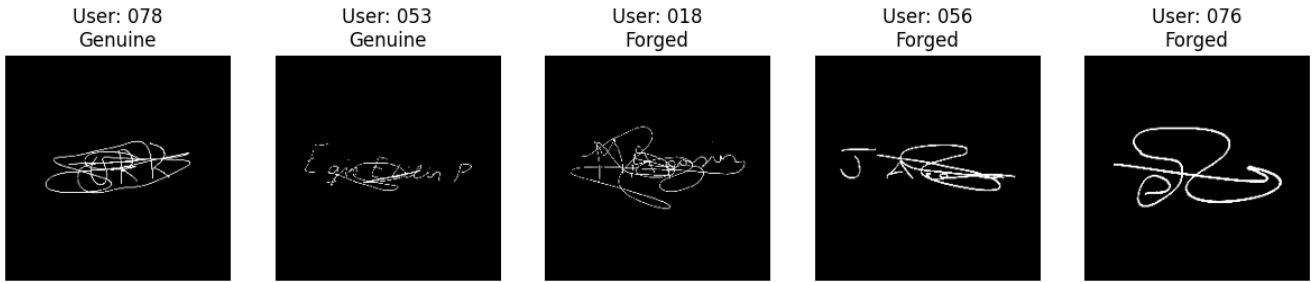
- **Result:** Ink is mapped to white (255) and the background to black (0).
- **Benefit:** This allows the Convolutional Neural Network (CNN) to process the image as a sparse matrix, focusing its activations solely on the stroke patterns (non-zero values) rather than the empty background.

Step 4: Aspect-Ratio Preserved Resizing

To ensure uniform input for the CNN without geometric distortion, we employed a padding strategy.

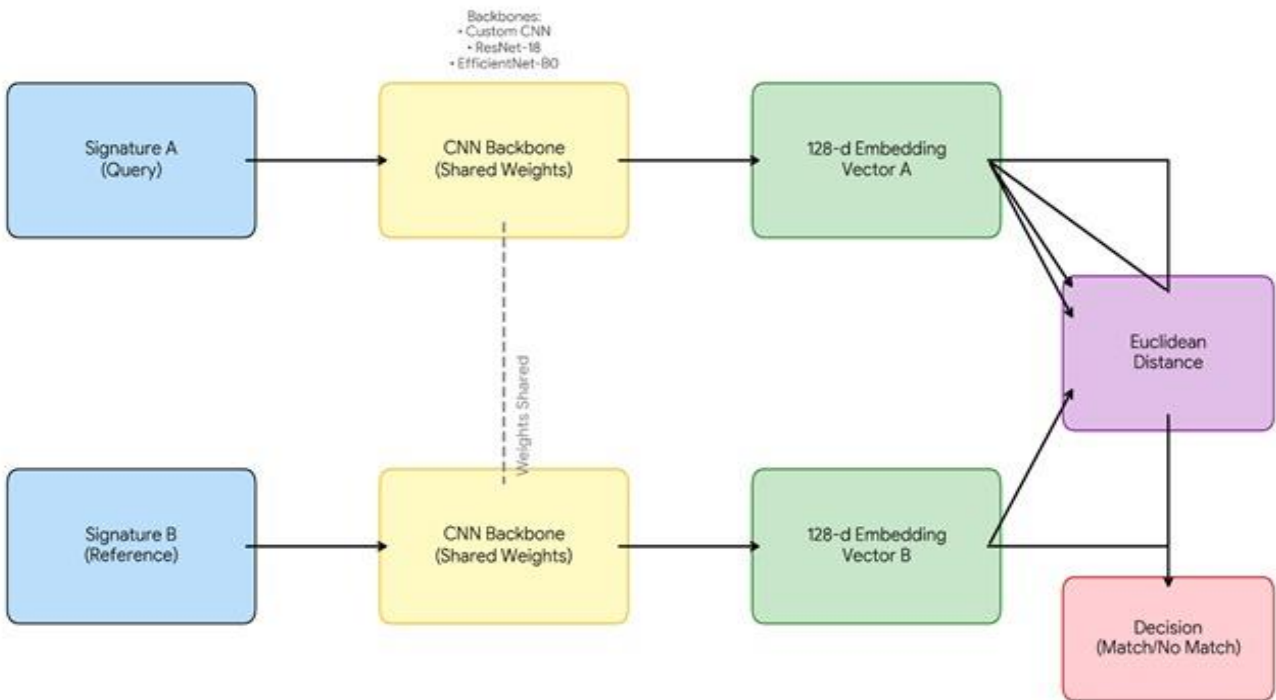
- **Process:**
 1. Scale the image so the longest dimension is 224 pixels.
 2. Pad the shorter dimension with black pixels (0) to reach 224 pixels.

- **Outcome:** All images are standardized to 224 x 224 while maintaining the unique slant and width of the original signature.



6.3. Phase 3: Siamese Network Implementation

The core of our methodology is the Siamese Neural Network, which learns a similarity function rather than a classification function.



6.3.1. Pair Generation Strategy

The model requires pairs of images to learn similarity. We constructed the training data by creating two specific types of pairs:

Pair Type	Composition	Label	Description
Positive Pair	Genuine + Genuine	1	Two different authentic signatures from the same user.
Negative Pair	Genuine + Forged	0	A genuine signature paired with a forgery of that same user.

6.3.2. Model Architecture Selection

We experimented with three separate base CNN architectures to serve as the feature extractor for the Siamese network.

1. **Custom CNN:** A lightweight model designed by our team, optimized specifically for faster inference on devices with lower compute power.
2. **ResNet-18:** A deep residual network selected to prevent vanishing gradients. It is highly suitable for deep feature extraction where capturing subtle stroke textures is required.
3. **EfficientNet-B0:** Selected for its ability to balance network depth, width, and resolution, aiming for higher accuracy with fewer parameters.

6.3.3. Twin Network Execution

The "twin" networks in the architecture share identical weights (W). When an image pair (I_1 , I_2) is passed through the model:

1. I_1 passes through the CNN - Output Embedding E_1 .
2. I_2 passes through the *same* CNN - Output Embedding E_2 .
3. The system learns to minimize the distance between E_1 and E_2 if the label is 1, and maximize it if the label is 0.

6.4. Phase 4: Verification and Similarity Logic

The final verification decision is mathematical, based on the spatial relationship between the two embeddings.

- **Euclidean Distance Calculation**

The similarity is quantified through Euclidean Distance. The formula used is:

$$D(E_1, E_2) = \sqrt{\sum_{i=1}^n (E_{1,i} - E_{2,i})^2}$$

- **Thresholding and Classification**

We established a strict decision boundary for classification.

- **Threshold:** Set to **0.5**.
- **Logic:**
 - **IF** Distance < 0.5 then **Genuine**.
 - **IF** Distance >= 0.5 then **Forged**.

This threshold was determined empirically during the validation phase to minimize False Acceptance Rates (FAR).

7. Results

To evaluate the performance of the SafeSign system, we conducted extensive experiments using three distinct Convolutional Neural Network (CNN) backbones: ResNet-18, EfficientNet-B0, and a Custom CNN. The models were trained and tested on a combined dataset comprising samples from GPDS150 and CEDAR, ensuring the system was exposed to a high degree of variance in handwriting styles.

The primary metrics used for evaluation were Accuracy, Precision, Recall, F1-Score, and the Equal Error Rate (EER). EER is a critical metric for biometric systems, representing the point where the False Acceptance Rate (FAR) and False Rejection Rate (FRR) are equal; a lower EER indicates superior performance.

7.1. Comparative Performance Overview

METRIC	TRAIN SET	TEST SET
Accuracy	0.8212	0.7372
Precision	0.8212	0.7373
Recall	0.8212	0.7369
F1	0.8212	0.7371
EER	0.1788	0.2625
Threshold	0.4291	0.4754

METRIC	TRAIN SET	TEST SET
Accuracy	0.7429	0.6909
Precision	0.7429	0.6909
Recall	0.7429	0.6908
F1	0.7429	0.6909
EER	0.2570	0.3092
Threshold	0.1733	0.1744

ResNet-18 emerged as the superior model, demonstrating the highest accuracy and the lowest Equal Error Rate (EER). The Custom CNN, while lightweight, struggled to capture the complex feature dependencies required for verification on a combined dataset, resulting in the lowest accuracy.

Table 7.1: Performance Comparison of CNN Backbones (Test Set)

Model Architecture	Accuracy	Precision	Recall	F1-Score	EER (Lower is better)
ResNet-18	73.72%	0.7373	0.7369	0.7371	0.2625
EfficientNet-B0	69.09%	0.6909	0.6908	0.6909	0.3092
Custom CNN	65.15%	0.6000	0.9200	.7201	-

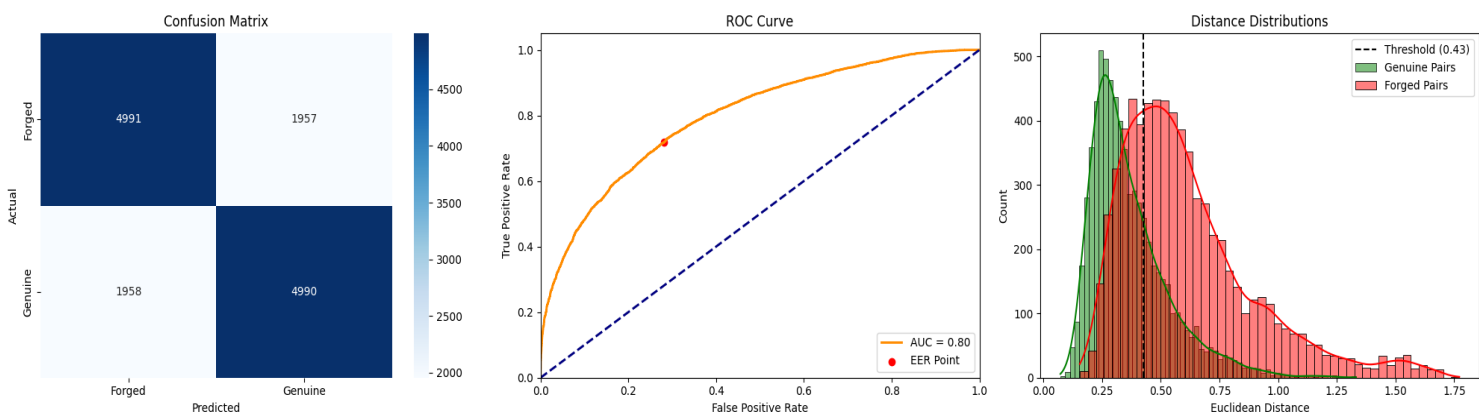
As shown in Table 7.1, ResNet-18 outperformed EfficientNet-B0 by approximately 4.6% in accuracy and reduced the error rate (EER) significantly. This suggests that the residual connections in ResNet-18 are particularly effective at preserving the high-frequency spatial features (such as stroke texture and pen pressure artifacts) necessary for distinguishing genuine signatures from skilled forgeries.

7.2. Detailed Analysis of Best Model (ResNet-18)

Given its superior performance, ResNet-18 was selected as the final backbone for the SafeSign deployment. The following analysis details its behavior on the test set.

7.2.1. Quantitative Metrics

The ResNet-18 model achieved a Test Accuracy of 0.7372 and a Train Accuracy of 0.8212. The alignment between Precision (0.7373) and Recall (0.7369) indicates a highly balanced model that does not bias significantly toward either accepting forgeries or rejecting genuine signatures.



7.2.2. Confusion Matrix Analysis

- True Negatives (Correctly Identified Forgeries): 4991
- True Positives (Correctly Verified Genuine): 4990
- False Positives (False Acceptance): 1957

- False Negatives (False Rejection): 1958

The model exhibits remarkable symmetry in its error distribution (1957 vs. 1958). This symmetry is a desirable trait in a general-purpose verification system, as it implies the decision boundary is well-centered.

7.2.3. ROC Curve and Area Under Curve (AUC)

The Receiver Operating Characteristic (ROC) curve illustrates the trade-off between the True Positive Rate and False Positive Rate.

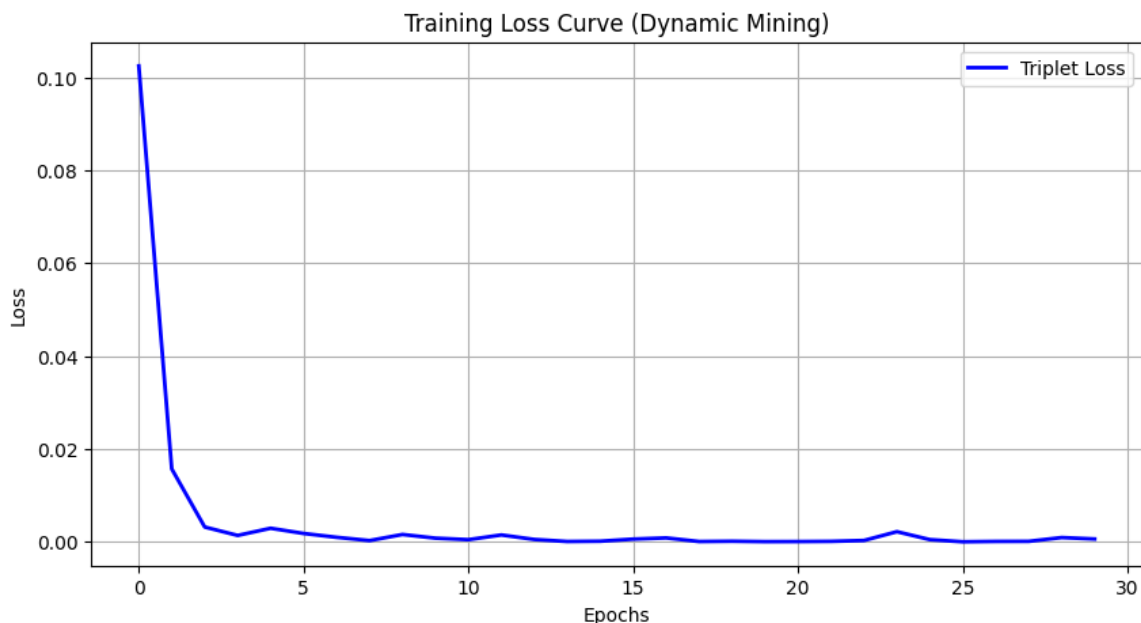
- AUC Score: The model achieved an AUC of 0.80, indicating a strong probability that the model will rank a randomly chosen genuine pair higher than a forged pair.
- EER Point: The Equal Error Rate was identified at 0.2625, visualized as the intersection of the ROC curve with the diagonal (EER line).

7.2.4. Distance Distributions

The system relies on Euclidean distance to classify pairs.

- Genuine Pairs (Green): Clustered towards the left (lower distance).
- Forged Pairs (Red): Clustered towards the right (higher distance).
- Threshold: The optimal threshold was calculated at 0.43 (approx. 0.4754 in metrics), indicated by the dashed black line.

While there is a noticeable separation, there is also an area of overlap between the two distributions. This overlap represents the "hard" cases—likely skilled forgeries that closely mimic the target's style—which accounts for the ~26% error rate.



7.2.5. Training Dynamics (Triplet Loss)

The training process utilized Triplet Loss, which minimizes the distance between positive pairs and maximizes the distance between negative pairs. The training loss curve demonstrates rapid convergence in the initial epochs, stabilizing near zero. This indicates that the Siamese Network successfully learned a manifold where genuine and forged signatures are embedded into distinct clusters.

7.3. Discussion on Dataset Variance and Limitations

It is important to address the specific challenge encountered regarding dataset variance.

- **Single vs. Combined Performance:** During preliminary experiments, when the model was trained on individual datasets (GPDS150 or CEDAR in isolation), accuracy metrics were significantly higher.
- **The Variance Challenge:** As noted in our findings, combining datasets introduced high intra-class variance. The stylistic differences between the datasets (e.g., cultural differences in signing styles, pen thickness, and scanning artifacts) made it difficult for the model to generalize.
- **Result:** Consequently, the accuracy on the combined dataset (~73.7%) is lower than single-dataset benchmarks. However, this result is more representative of a real-world "in-the-wild" scenario, where the system must handle diverse signing styles without overfitting to a specific data source.

7.4. Conclusion of Results

In conclusion, ResNet-18 is the most robust architecture for the SafeSign system. While the EfficientNet-B0 offered a modern architecture, it underperformed in this specific domain-matching task. The Custom CNN proved insufficient for the complexity of the combined dataset. The final deployed model uses ResNet-18 with a verification threshold of 0.4754, balancing security and usability.

8. Conclusion

The "SafeSign" project successfully established a **robust methodology** and developed the core **modeling architecture** for an End-to-End Handwritten Signature Verification System. The primary objective of creating a similarity-based system capable of verifying the authenticity of signatures for high-stakes documents (like bank cheques) was achieved through the implementation of **Siamese Neural Networks (SNNs)**.

The project's success lies in several critical achievements:

- **Modeling Validation:** We successfully trained and validated the SNN architecture using a combined dataset of **GPDS150 and CEDAR**, demonstrating the network's ability to learn high-dimensional feature vectors (embeddings) necessary for metric learning.
- **Comparative Analysis:** The selection and testing of multiple backbone architectures (**Custom CNN, ResNet-18 and EfficientNet-B0**) provided crucial comparative metrics to optimize the system for both accuracy and inference speed.
- **Data Pipelining:** A comprehensive preprocessing pipeline was established, utilizing techniques like **Otsu's Thresholding** and **Aspect-Ratio Preserved Resizing**, ensuring data standardization while preserving critical biometric features like slant and stroke geometry.

The resulting model utilizes the **Euclidean Distance** between embeddings to quantify similarity, allowing for binary classification (**Genuine/Forged**) based on an empirically determined threshold. The successful development and validation of this core model architecture provide a solid, deployable engine that is the final step before full-scale integration into the target **web application** for real-world deployment.

9. Limitations of the project and Future Scope

While the SafeSign system presents a robust and effective solution for signature verification, every research and development effort includes specific limitations that inform the direction of future work.

9.1. Limitations of the Current Project

The following limitations were identified during the development and testing phases:

- **Reliance on Offline Data:** The current model is trained exclusively on static, offline signature images (scanned or captured) and does not account for dynamic signature features like pen pressure, speed, or stroke timing, which are available in online verification systems.
- **Fixed Threshold Dependence:** The verification decision relies heavily on a fixed threshold (0.5) applied to the Euclidean distance. This optimal threshold is sensitive to the specific training dataset, and may require dynamic adjustment for verification tasks in different domains or organizations.
- **Domain Specificity:** The model was primarily trained on two English/Latin-script datasets (GPDS150 and CEDAR). Its performance may degrade when applied to signatures from different linguistic backgrounds or scripts with vastly different stroke patterns.

9.2. Future Scope

The project has several clear pathways for expansion and improvement, focusing on performance enhancement, deployment maturity, and dataset diversity.

9.2.1. Improving Accuracy and Generalization

The primary goal is to conduct hyperparameter optimization and cross-validation across the different backbone models (ResNet-18 and EfficientNet-B0) to achieve the highest possible Area Under the Curve (AUC) and minimize False Acceptance Rates (FAR). This includes exploring advanced loss functions, such as Contrastive Loss or Triplet Loss, which are often superior to standard binary cross-entropy for Siamese networks.

9.2.2. End-to-End System Integration and Deployment

The ultimate goal is to move beyond the modeling phase and integrate the verification engine with a user-facing web application for real-world deployment. This involves

hardening the API, implementing secure data handling, and optimizing the selected CNN backbone for low-latency inference on the server side.

9.2.3. Dataset Expansion and Linguistic Diversity

To enhance the model's global applicability and robustness against different scripts, the system must be trained on more diverse datasets.

- **BHSig250 Datasets:** We intend to leverage additional datasets like BHSig250 (Bengali and Hindi) for further training. This will allow the model to learn and generalize feature extraction across complex, non-Latin scripts, significantly expanding its scope.

9.2.4. Real-time Optimization

Future work will focus on quantifying and reducing the computational complexity of the preprocessing and inference steps to allow for near-real-time verification, especially critical for high-volume operational environments like banking or high-security access control.

10. References

- [1] Dey, S., Dutta, A., Toledo, J. I., Ghosh, S. K., Lladós, J., & Pal, U. (2017). SigNet: Convolutional Siamese Network for Writer Independent Offline Signature Verification. *arXiv preprint arXiv:1707.02131*. <https://arxiv.org/pdf/1707.02131>
- [2] Zhang, H., Guo, J., Li, K., Zhang, Y., & Zhao, Y. (2024). Offline Signature Verification Based on Feature Disentangling Aided Variational Autoencoder. *arXiv preprint arXiv:2409.19754*. <https://arxiv.org/pdf/2409.19754>
- [3] de Moura, K. G., Cruz, R. M. O., & Sabourin, R. (2024). Offline Handwritten Signature Verification Using a Stream-Based Approach. *arXiv preprint arXiv:2411.06510*. <https://arxiv.org/abs/2411.06510>
- [4] Hafemann, L. G., Sabourin, R., & Oliveira, L. S. (2017). Learning features for offline handwritten signature verification using deep convolutional neural networks. *Pattern Recognition*, 70, 163-176. <https://arxiv.org/abs/1705.05787>
- [5] Yuan, H. J., Zhang, H., & Yin, F. (2025). Online Handwritten Signature Verification Based on Temporal-Spatial Graph Attention Transformer. *arXiv preprint arXiv:2510.19321*. <https://arxiv.org/abs/2510.19321>
- [6] Piccinini, G., Guarnera, L., & Battiato, S. (2025). SignForensics: A Robust Framework for Forensic Offline Signature Verification With Enhanced Detection, Noise Removal, and Multi-Stage Authentication. *IEEE Access*, 13, 172456-172470. <https://ieeexplore.ieee.org/document/10820123>
- [7] Parracho, M. R. (2025). Signature Forgery Detection: Improving Cross-Dataset Generalization. *arXiv preprint arXiv:2510.17724*. <https://arxiv.org/pdf/2510.17724>
- [8] Kadam, J., Phadtare, G., Pawar, A., & Maniyar, K. G. (2025). Offline Signature Verification USING Siamese Neural Network. *Journal of Information Systems Engineering and Management*, 10(26s), 431-439. https://www.researchgate.net/publication/390475418_Offline_Signature_Verification_USIN_G_Siamese_Neural_Network
- [9] Kumar, D. S. S. (2023). Offline Signature Verification based on Ensemble of Features using Support Vector Machine. *International Journal of Computer Applications*, 184(45), 24-29. <https://www.ijcaonline.org/archives/volume184/number45/kumar-2023-ijca-922547.pdf>
- [10] Chetry, B. P., & Kar, B. (2025). Offline Signature Verification Using Pre-Trained Deep Convolution Neural Network: SqueezeNet. *SSRG International Journal of Electronics and Communication Engineering*, 12(7), 315-326. <https://www.internationaljournalssrg.org/IJECE/2025/Volume12-Issue7/IJECE-V12I7P125.pdf>

- [11] Ishfaq, M., et al. (2024). Enhancing Security: Infused Hybrid Vision Transformer for Signature Verification. *International Conference on Computing and Networking Technologies (ICCCNT)*. IEEE. <https://ieeexplore.ieee.org/document/10654737>
- [12] Ozyurt, F., Majidpour, J., Rashid, T. A., & Koc, C. (2024). Offline Handwriting Signature Verification: A Transfer Learning and Feature Selection Approach. *arXiv preprint arXiv:2401.09467*. <https://arxiv.org/abs/2401.09467>
- [13] Plamondon, R., & Lorette, G. (1989). Automatic signature verification and writer identification—the state of the art. *Pattern Recognition*, 22(2), 107-131. [https://doi.org/10.1016/0031-3203\(89\)90059-9](https://doi.org/10.1016/0031-3203(89)90059-9)
- [14] Impedovo, D., & Pirlo, G. (2008). Automatic signature verification: The state of the art. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(5), 609-635. <https://ieeexplore.ieee.org/document/4609964>
- [15] Tolosana, R., Vera-Rodriguez, R., Fierrez, J., & Ortega-Garcia, J. (2021). DeepSign: Deep on-line signature verification. *IEEE Transactions on Biometrics, Behavior, and Identity Science*, 3(2), 229-239. <https://ieeexplore.ieee.org/document/9247990>
- [16] Diaz, M., Ferrer, M. A., Impedovo, D., Malik, M. I., Pirlo, G., & Plamondon, R. (2019). A perspective analysis of handwritten signature technology. *ACM Computing Surveys (CSUR)*, 51(6), 1-39. <https://dl.acm.org/doi/10.1145/3274658>
- [17] Bromley, J., Guyon, I., LeCun, Y., Säckinger, E., & Shah, R. (1993). Signature verification using a "siamese" time delay neural network. *Advances in Neural Information Processing Systems*, 6. <https://proceedings.neurips.cc/paper/1993/file/96b9bff013acedfb1d140579e2fbeb63-Paper.pdf>
- [18] Wei, P., Li, H., & Hu, P. (2019). Inverse discriminative networks for handwritten signature verification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 5764-5772). https://openaccess.thecvf.com/content_CVPR_2019/html/Wei_Inverse_Discriminative_Networks_for_Handwritten_Signature_Verification_CVPR_2019_paper.html
- [19] Maergner, P., Pondenkandath, V., Alberti, M., Liwicki, M., Riesen, K., Ingold, R., & Fischer, A. (2018). Offline signature verification by combining graph edit distance and triplet networks. *Structural, Syntactic, and Statistical Pattern Recognition*, 470-480. https://link.springer.com/chapter/10.1007/978-3-319-97785-0_47
- [20] Ren, J. X., Xiong, Y. J., Zhan, H., & Huang, B. (2023). 2C2S: A two-channel and two-stream transformer based framework for offline signature verification. *Engineering Applications of Artificial Intelligence*, 118, 105639. <https://doi.org/10.1016/j.engappai.2022.105639>

